



Rapport de Stage

Présenté par :

Naoufal Benamar

Encadré par :

M. Nadi Tomeh

Tutrice pédagogique :

Mme Céline Rouveirol

Année universitaire : 2025 – 2026

Engagement de non-plagiat

Je, soussigné Naoufal Benamar, étudiant en 3eme année d'école d'ingénieur à Sup Galilée, déclare être pleinement conscient que la copie de tout ou partie d'un document, quel qu'il soit, publié sur tout support existant, y compris sur Internet, constitue une violation du droit d'auteur ainsi qu'une fraude caractérisée. En conséquence, je déclare que mon mémoire ne comporte aucun plagiat, et assure que j'ai cité explicitement, à chaque fois que j'en ai fait usage, toutes les sources utilisées pour le rédiger.

Fait à Villetaneuse , le 15/09/2025

Signature :

A handwritten signature in black ink, appearing to read 'Naoufal Benamar', written over a horizontal line.

Remerciements

Je tiens tout d'abord à exprimer ma profonde gratitude au **Laboratoire d'Informatique de Paris-Nord (LIPN)** pour m'avoir accueilli durant ce stage et m'avoir permis de m'intégrer dans un environnement de recherche stimulant et enrichissant.

Je remercie en particulier **M. Nadi Tomeh**, maître de conférences et encadrant de mon stage, pour sa disponibilité, ses conseils avisés et son accompagnement tout au long de ce travail. Son expertise et ses orientations m'ont grandement aidé à avancer dans mes recherches et à mieux comprendre les enjeux scientifiques liés à la classification multi-label extrême.

Je souhaite également adresser mes remerciements les plus sincères à tous les professeurs de deuxième année INFO2. Grâce à leur enseignement de qualité et à leur soutien constant, j'ai pu acquérir les compétences nécessaires pour mener à bien cette année, ainsi que ce stage. Leur pédagogie a joué un rôle essentiel dans mon parcours, me permettant ainsi d'en tirer un grand bénéfice personnel et académique.

Table des matières

INTRODUCTION.....	5
I. PRESENTATION DU LIPN.....	6
II. PRESENTATION DE L'EQUIPE (RCLN)	7
III. SUJET, CONTEXTE ET BUT DU STAGE	8
IV. OUTILS UTILISES	9
V. PRINCIPE DE FONCTIONNEMENT DU MODELE XR-TRANSFORMER.....	9
VI. TRAVAIL REALISE	13
1. <i>Première approche : échantillonnage basé sur la fréquence des labels</i>	14
2. <i>Deuxième approche : échantillonnage basé sur le transport optimal</i>	20
VII. CE QUE CE STAGE M'A APPORTE.....	22
CONCLUSION.....	23
REFERENCES	23

Introduction.

Durant les mois de mai, juin, juillet et août, j'ai effectué un stage de quatre mois au Laboratoire d'Informatique de Paris-Nord (LIPN), au sein de l'équipe RCLN (Représentation des Connaissances et Langage Naturel), sous la direction de M. Nadi Tomeh, maître de conférences.

J'ai choisi ce stage car je m'intéresse particulièrement à l'apprentissage automatique (machine learning) et je souhaitais approfondir mes connaissances dans ce domaine à travers un projet de recherche appliquée. Par ailleurs, dans le cadre de ma formation, il y a un certain nombre de semaines de stage en entreprise à respecter, que je réaliserai lors de mon stage de fin d'études. Ce stage de deuxième année au LIPN est donc une opportunité complémentaire, davantage orientée vers la recherche académique.

Étant à la fois intéressé par la recherche et encore indécis quant à mon avenir professionnel, partagé entre une carrière académique et un futur en entreprise, j'ai voulu profiter de cette expérience pour découvrir concrètement le fonctionnement d'un laboratoire de recherche. Cette immersion m'a permis d'observer et de participer à des travaux scientifiques, afin de mieux comprendre ce qu'implique une carrière dans la recherche, et ainsi pouvoir comparer cette voie avec celle du monde industriel pour orienter mes choix futurs de manière éclairée et réfléchie.

I. Présentation du LIPN

Le Laboratoire d'Informatique de Paris-Nord (LIPN) est une unité mixte de recherche (UMR 7030) entre le CNRS et l'Université Paris XIII, dite Université Sorbonne Paris Nord (USPN).

L'ambition du LIPN est d'appréhender les grands problèmes scientifiques depuis la recherche fondamentale jusqu'à la valorisation des résultats obtenus et de développer ses activités de recherche en favorisant les collaborations scientifiques et l'interdisciplinarité avec l'environnement proche.

Les principales thématiques de l'unité, organisée autour de cinq équipes de recherche réunissant 150 membres, sont l'apprentissage automatique, l'optimisation combinatoire et le calcul haute performance, la conception et l'analyse de modèles combinatoires à l'interface de la physique et de l'algorithmique, les fondements du calcul et la vérification formelle, le traitement automatique du langage naturel et la représentation des connaissances.



Figure : Logo du LIPN

Pour mener à bien ses recherches, le LIPN est constitué de 6 équipes de recherches :

- A3 : Apprentissage Artificiel et Applications
- AOC : Algorithmes et Optimisation Combinatoire
- CALIN : Combinatoire, ALgorithmique et INteractions
- LoCal : Logique et Calcul
- RCLN : Représentation des Connaissances et Langage Naturel
- SAFER : Safety and Security Analyses via Formal and Efficient veRification

Il y a aussi les équipes Administrative et Informatique, qui sont là pour assurer le bon déroulement des activités du laboratoire.

Les recherches menées par les membres du LIPN ont donné lieu à quelques milliers de publications.

II. Présentation de l'équipe (RCLN)

Responsable : Nathalie Pernelle

L'équipe rassemble des compétences en Traitement Automatique des Langues (TAL), en linguistique de corpus, en web sémantique, ainsi qu'en fouille de données et en apprentissage automatique. Ces compétences complémentaires lui confèrent un positionnement original et lui permettent de mener des travaux innovants pour l'analyse et l'exploration de corpus de textes, ainsi que pour l'acquisition de connaissances à partir de textes ou de graphes de connaissances.

L'équipe travaille sur des approches permettant de traiter la complexité des problèmes dans les textes ou dans les graphes de connaissances, comme l'analyse syntaxique profonde, l'extraction simultanée de relations sémantiques et d'entités nommées, ou encore la complétion de graphes, à l'aide de méthodes issues de l'apprentissage profond, de l'optimisation combinatoire, de l'exploration de données ou de la programmation logique inductive. L'équipe est notamment active sur le traitement de langues sous-dotées ou spécialisées, la détection de néologismes et l'analyse des microblogs.

L'équipe RCLN prend une part active dans la direction et les travaux du Labex « Empirical Foundations of Linguistics » (EFL), dont elle coordonne l'axe Analyse sémantique computationnelle. Au niveau local elle est membre de la fédération MathSTIC dont elle co-anime l'axe Optimisation et apprentissage appliqués aux contenus numériques.

L'équipe est structurée autour de trois axes de recherche étroitement liés :

- Algorithmes pour l'analyse syntaxique et sémantique des textes
- Exploration et génération de textes
- Acquisition de connaissances.

Ces trois axes sont complémentaires : l'analyse syntactico-sémantique de corpus peut servir de base à l'exploration de textes et à l'acquisition de connaissances, et inversement les algorithmes d'analyse peuvent tirer profit des connaissances acquises.

La fouille de littérature scientifique est définie comme un axe transversal de l'équipe. En effet, les publications de recherche ou les jeux de données partagés pourraient être mieux exploités par des systèmes intelligents afin de soutenir et d'accélérer les efforts scientifiques, en facilitant l'extraction d'experts, la génération de textes à l'état de l'art, la génération d'hypothèses scientifiques, ou encore pour soutenir ou contredire certaines hypothèses.

Mon encadrant M. Nadi Tomeh est, quant à lui, maître de conférences, spécialisé dans le traitement automatique du langage naturel, et activement impliqué dans les recherches mêlant apprentissage automatique et TAL.

III. Sujet, contexte et but du stage

Le stage que j'ai effectué porte sur le thème : « **Apprentissage par Curriculum et Meta-Learning pour la Classification Multi-Label Extrême (XMLC)** ».

La classification multi-label extrême (XMLC) est une tâche d'apprentissage supervisé qui consiste à associer à un document un ensemble d'étiquettes parmi un très grand nombre de catégories possibles. Contrairement à la classification classique où chaque document est rattaché à une seule étiquette, la classification multi-label permet de refléter la complexité des données réelles, puisqu'un même document peut appartenir simultanément à plusieurs catégories. On parle d'extrême car le nombre d'étiquettes est très élevé, souvent de l'ordre de dizaines voire centaines de milliers, ce qui pose des défis de scalabilité, de déséquilibre entre étiquettes fréquentes et rares, et de performance des algorithmes.

Cette approche trouve des applications dans divers domaines :

- Classification d'articles scientifiques : un même article peut être étiqueté avec plusieurs thématiques (par exemple apprentissage automatique, traitement du langage naturel et optimisation).
- Systèmes de recommandation : un film peut être associé à plusieurs genres (action, science-fiction, aventure).
- Recherche d'information : les documents ou pages web peuvent être indexés avec plusieurs mots-clés ou thématiques pour améliorer la pertinence des résultats.
- Analyse de brevets : un brevet peut couvrir plusieurs domaines techniques.
- Modération de contenus en ligne : un même message peut être simultanément étiqueté comme publicité, désinformation et contenu sensible.

Pour répondre à ces défis, deux pistes principales ont été explorées dans ce stage :

- L'apprentissage par curriculum, qui consiste à organiser l'entraînement de façon progressive. On commence par des tâches simples, liées aux étiquettes fréquentes, puis l'on introduit progressivement des tâches plus complexes, correspondant aux étiquettes rares. Cette approche vise à améliorer la robustesse et la généralisation des modèles.
- Le meta-learning, qui permet au modèle d'apprendre à apprendre. L'objectif est de lui donner la capacité de s'adapter rapidement à de nouvelles tâches ou à des étiquettes rares, en tirant parti de l'expérience acquise sur d'autres ensembles d'étiquettes.

Les questions de recherche abordées au cours de ce travail sont les suivantes :

- Comment définir et ordonner efficacement un curriculum allant des étiquettes fréquentes aux plus rares ?
- Dans quelle mesure le meta-learning peut-il faciliter l'adaptation rapide aux étiquettes rares ?
- Quelle est l'importance de modéliser les dépendances sémantiques ou structurelles entre les étiquettes ?
- Comment garantir que ces approches restent scalables et applicables à de grands ensembles de données ?

IV. Outils utilisés

Au cours de ce stage, voici les différents outils matériels et logiciels que j'ai utilisé :

- Infrastructure matérielle : serveur du LIPN équipé d'une carte graphique NVIDIA A40, facilitant l'entraînement de modèles à grande échelle. L'accès et la gestion des ressources de calcul se faisaient via le gestionnaire de tâches Slurm.
- Modèle de base : j'ai travaillé principalement avec le modèle XR-Transformer, fourni dans la bibliothèque PECOS développée par Amazon. Ce modèle a été retenu car il s'avère plus rapide et plus performant en termes de métriques que la majorité des autres modèles de prédiction en XMLC. Initialement, l'utilisation d'un autre modèle, LightXML, avait été envisagée. Cependant, ce dernier repose sur une ancienne version de PyTorch et nécessite l'extension Apex de NVIDIA, ce qui n'était pas adapté à notre infrastructure. De plus, LightXML étant quelque peu daté, XR-Transformer constituait une alternative plus robuste et plus adaptée à nos contraintes techniques.
- Jeu de données (dataset) : les expérimentations ont été réalisées sur le dataset AmazonCat-13K, construit à partir des métadonnées de produits de la plateforme Amazon. Ce dataset regroupe environ 1,18 million de documents (titres et descriptions de produits), associés à 13 330 catégories distinctes.

Chaque produit peut être rattaché à plusieurs catégories simultanément (d'où le caractère multi-label), ce qui reflète bien la complexité des données réelles. Par exemple, un livre peut être associé à la fois aux catégories Informatique, Science des données et Apprentissage automatique.

- Langage et bibliothèques : toute l'implémentation et l'expérimentation ont été réalisées en Python, avec l'appui de bibliothèques standards pour le machine learning et le NLP, telles que TensorFlow, NumPy, scikit-learn et PyTorch.

Maintenant que nous avons abordé les principaux outils utilisés, j'aimerais que l'on s'attarde un peu plus sur le principe de fonctionnement du modèle en lui-même. Cela sera crucial afin de comprendre la logique avec laquelle il fonctionne et ainsi pouvoir mieux répondre à nos questions initiales.

V. Principe de fonctionnement du modèle XR-Transformer

Le modèle XR-Transformer, disponible dans la bibliothèque PECOS développée par Amazon, est un modèle de classification multi-label extrême qui s'appuie sur les architectures de type transformer. L'idée générale est de combiner des techniques innovantes (que l'on verra plus tard) de représentation du texte avec des méthodes efficaces (que l'on verra plus tard également) pour gérer un très grand nombre d'étiquettes.

Voici les étapes principales du traitement de texte par notre modèle :

1. Tokenisation du texte

Un document (par exemple le titre ou la description d'un produit) est d'abord découpé en petites unités appelées tokens. Cela peut être des mots entiers, ou bien des morceaux de mots pour les termes rares. Cette étape est essentielle car le modèle ne peut pas comprendre directement le texte brut : il a besoin de le transformer en éléments plus simples et répétitifs pour l'analyser correctement.

2. Représentations vectorielles (embeddings)

Une fois découpés, les tokens sont transformés en vecteurs numériques, appelés embeddings. Chaque vecteur contient des informations sur le mot lui-même et sur son sens dans le contexte. Par exemple, le mot *banc* n'aura pas la même représentation dans *banc public* que dans *banc de poissons*. Dans XR-Transformer, ces représentations s'appuient sur des méthodes modernes comme BERT, ce qui permet d'obtenir des vecteurs adaptés et sensibles au contexte du texte.

3. Apprentissage sémantique avec les Transformers

Le cœur du modèle repose sur le mécanisme d'attention, qui sert à identifier quels mots du texte sont les plus importants par rapport aux autres.

Avec les anciens modèles comme les RNN et LSTM, le texte était lu mot par mot, de gauche à droite. C'était lent et ces modèles avaient du mal à relier correctement les informations entre le début et la fin d'un texte.

Les transformers, eux, regardent tous les mots en même temps et en parallèle. Cela permet à chaque mot de se connecter directement aux autres, rendant l'analyse à la fois plus rapide et plus précise.

En résumé, les transformers comprennent mieux le sens global d'un document, ce qui est très utile pour des tâches complexes comme la classification multi-label extrême.

4. Indexation et prédiction des étiquettes

En XMLC, il existe un très grand nombre d'étiquettes possibles (parfois des dizaines de milliers). Le modèle XR-Transformer ne peut pas simplement comparer un document à toutes ces étiquettes une par une. Pour être plus efficace, il suit plusieurs étapes :

- Il apprend d'abord une représentation vectorielle des étiquettes elles-mêmes (souvent à partir de leur nom ou description).
- Ensuite, il utilise un mécanisme de recherche rapide pour sélectionner seulement un petit groupe d'étiquettes candidates pertinentes pour le document.
- Enfin, une classification fine est appliquée pour choisir, parmi ces candidates, celles qui correspondent le mieux.

5. Avantages du XR-Transformer

En combinant des représentations contextuelles puissantes (grâce aux transformers et à BERT) et une stratégie d'indexation hiérarchique efficace, XR-Transformer parvient à obtenir de meilleurs résultats que beaucoup d'autres modèles en XMLC. Il est non seulement plus précis, mais aussi plus rapide et mieux adapté aux datasets très volumineux. C'est pour cette raison qu'il a été choisi comme base de mes expérimentations durant ce stage.

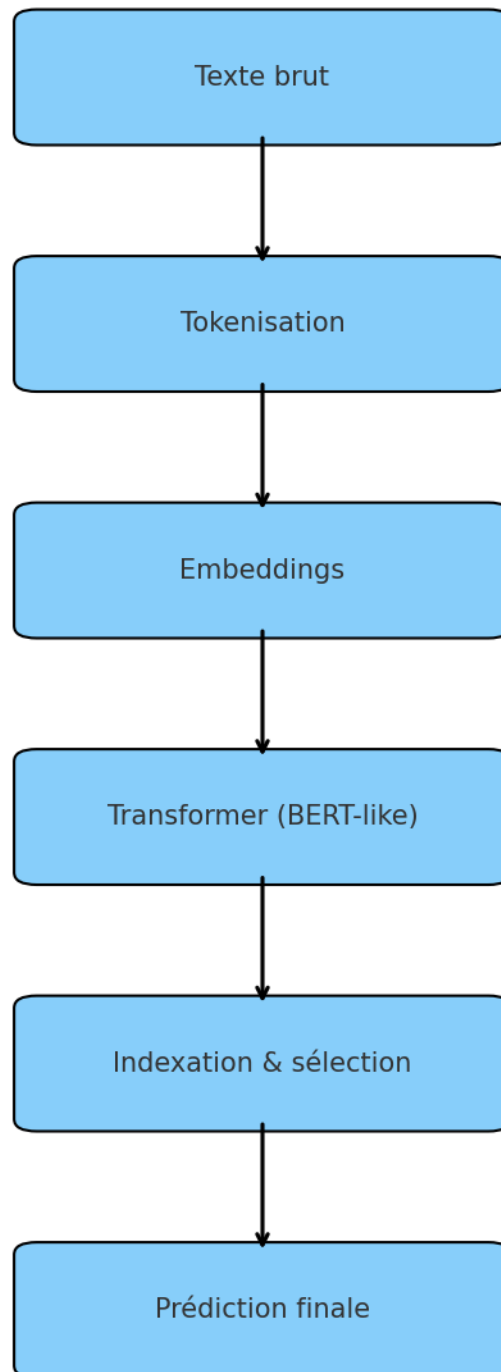


Figure : Principales étapes du traitement de texte par notre modèle

HLT (Hierarchical Label Tree) (Arbre hiérarchique de labels) et clustering :

```
08/08/2025 14:35:54 - INFO - __main__ - Loaded 1186239 training sequences
08/08/2025 14:36:13 - INFO - pecos.xmc.xtransformer.model - Hierarchical label tree: [8, 128, 13330]
08/08/2025 14:36:13 - INFO - pecos.xmc.xtransformer.model - Fine-tune Transformers with nr_labels=[8, 128]
08/08/2025 14:36:14 - INFO - pecos.xmc.xtransformer.model - Fine-tuning XR-Transformer with tfn at level 0, nr_labels=8, avr_M_nnz=8
```

Mais avant toute la partie de traitement du texte (tokenisation, embeddings, transformer), un clustering des labels est réalisé. Cette étape, faite en amont, sert à organiser les étiquettes de manière hiérarchique pour rendre la classification plus efficace.

En classification multi-label extrême, le principal défi est le nombre très élevé d'étiquettes à gérer. Dans le dataset AmazonCat-13K, on compte environ 13 330 labels. Si le modèle devait comparer chaque document à toutes ces étiquettes, le calcul serait trop long et trop coûteux.

Pour éviter cela, XR-Transformer utilise une structure appelée Hierarchical Label Tree (HLT). Le principe est de regrouper les étiquettes proches en clusters, afin que le modèle n'ait à chercher que dans les groupes pertinents au lieu de parcourir toutes les étiquettes.

Dans mon cas, l'organisation se fait en deux niveaux :

- Niveau 0 : les 13 330 labels sont regroupés en 8 clusters principaux ;
- Niveau 1 : chacun de ces clusters est subdivisé, pour donner au total 128 sous-clusters ;
- Enfin, à l'intérieur des sous-clusters, on retrouve les labels finaux.

Le regroupement est réalisé grâce à l'algorithme k-means, ou plus exactement une variante équilibrée (balanced k-means), qui garantit des clusters de taille similaire. Les labels sont représentés par des vecteurs numériques (embeddings) construits principalement à partir de leurs co-occurrences dans les documents. La proximité est donc définie de manière statistique : deux labels qui apparaissent souvent ensemble auront tendance à être placés dans le même cluster.

Lors de la classification, le modèle ne parcourt donc pas directement les 13 330 étiquettes. Il sélectionne d'abord le cluster pertinent au niveau 0, puis affine la recherche au niveau 1, avant de choisir les étiquettes finales.

Tout cela rend la classification à la fois plus rapide, plus scalable et plus précise, car le modèle se concentre uniquement sur les étiquettes réellement pertinentes.

Voici une illustration permettant de mieux visualiser la structure de notre HLT :

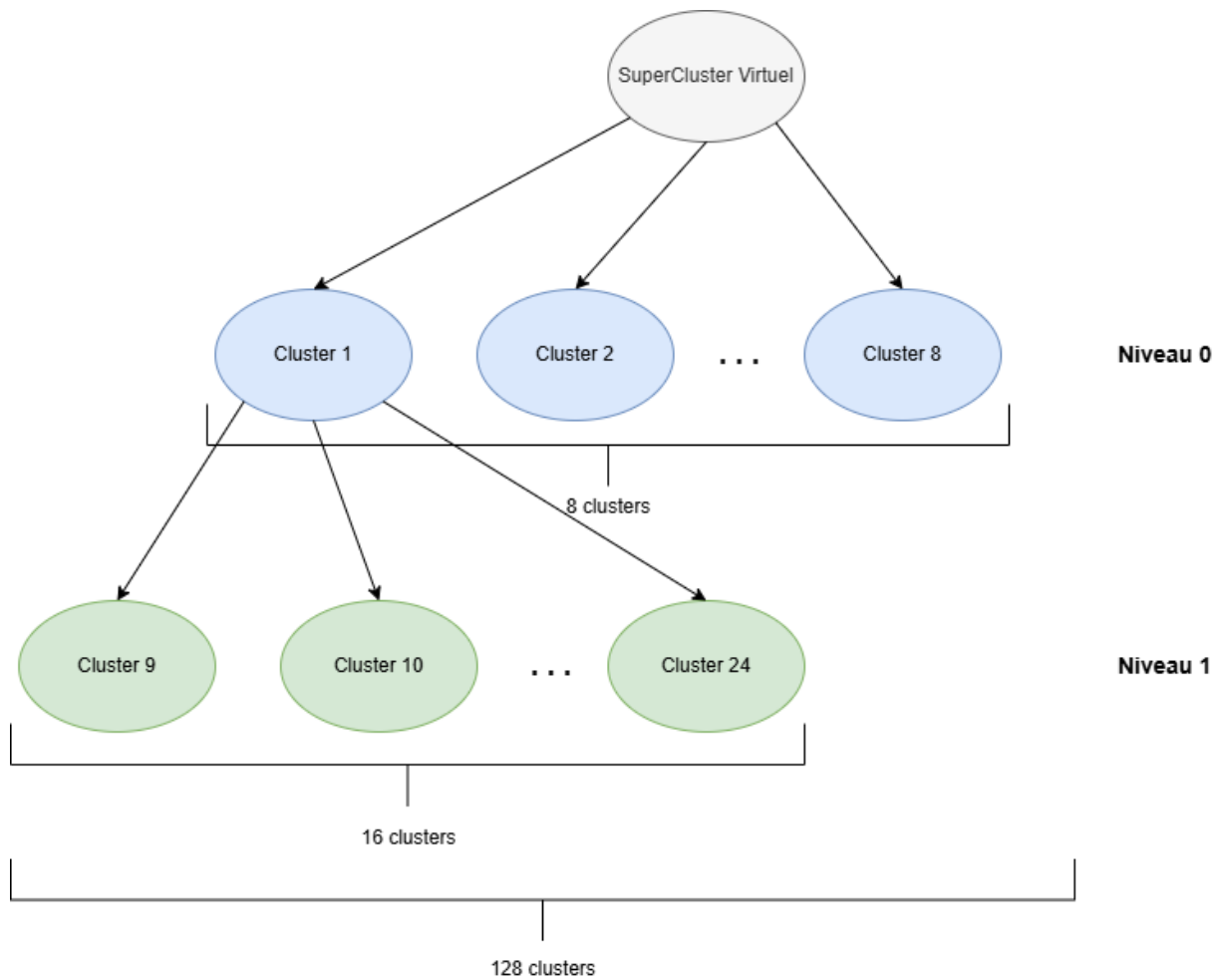


Figure : représentation schématique du HLT

VI. Travail réalisé

Le travail effectué au cours de ce stage s'est déroulé en deux grandes étapes. Dans un premier temps, nous avons cherché à affiner le modèle XR-Transformer en mettant en place un curriculum d'apprentissage basé sur la fréquence des labels : l'idée était de commencer l'entraînement par les étiquettes les plus fréquentes, puis d'introduire progressivement les plus rares, afin d'améliorer la robustesse du modèle face au déséquilibre des données. Dans un second temps, nous avons exploré une approche complémentaire basée sur le transport optimal, qui permet de prendre en compte non seulement la fréquence, mais aussi la proximité sémantique entre les labels. Cette méthode vise à mieux exploiter les relations entre étiquettes pour améliorer la qualité de la classification.

1. Première approche : échantillonnage basé sur la fréquence des labels

Avant de mettre en place un échantillonnage basé sur la fréquence, il a d'abord été nécessaire de constater le déséquilibre présent dans les données. Pour cela, nous avons réalisé une analyse de la distribution des classes, qui a montré que certaines étiquettes apparaissent très fréquemment tandis que d'autres ne disposent que de très peu d'exemples.

Voici les résultats de notre analyse :

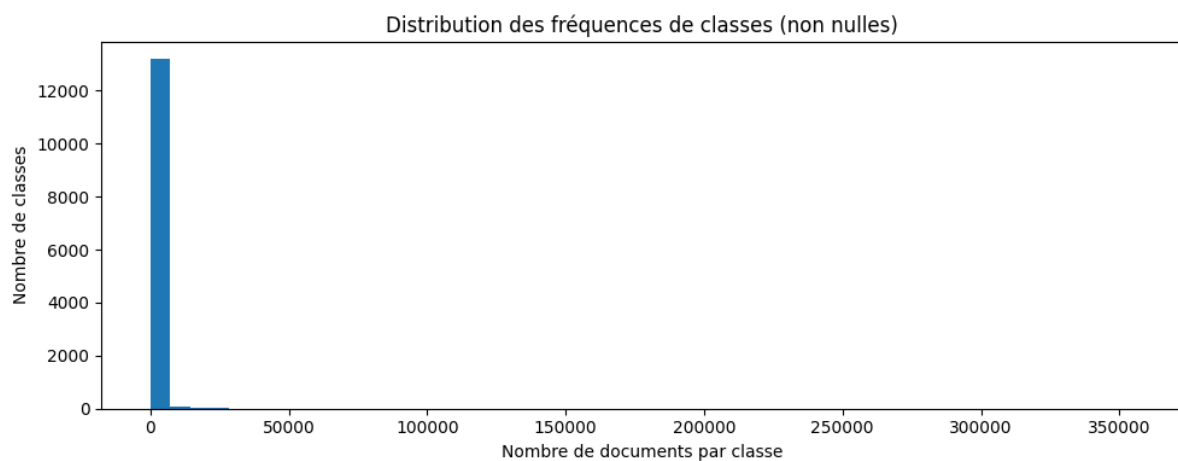
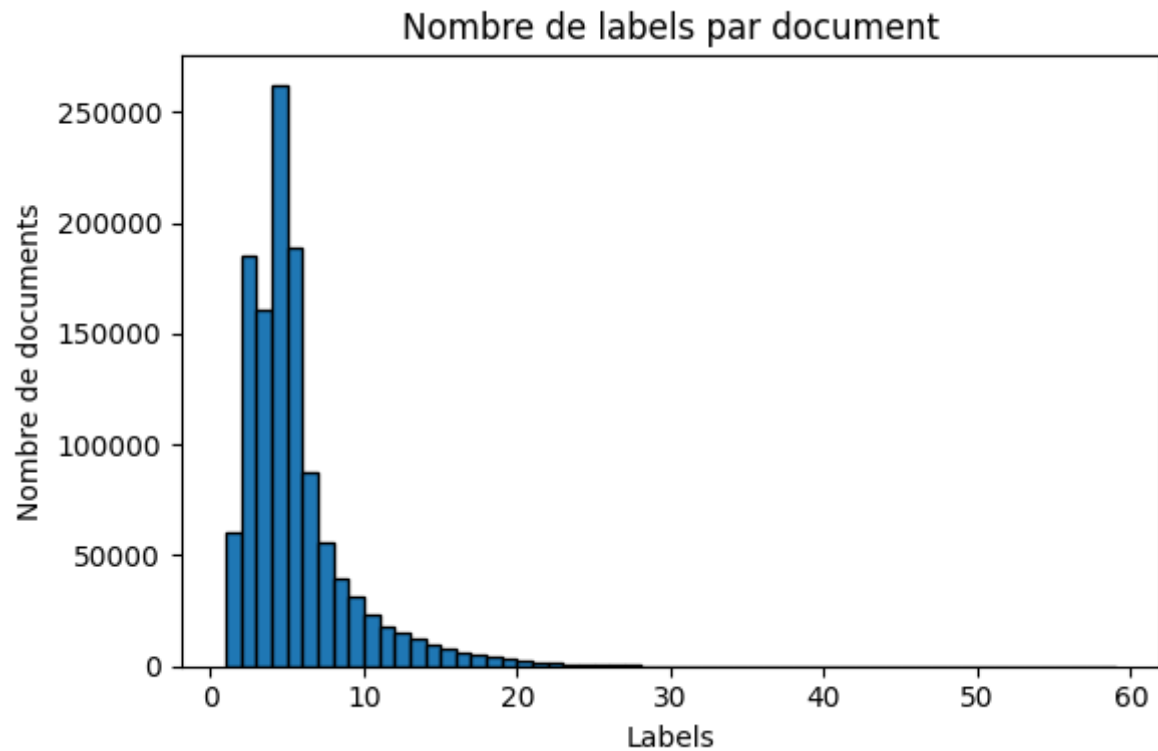
```
naben@Naoufal:~/StageXMLC/Datasets/script$ python3 analyse_amazoncat13k.py
📖 Lecture de trn.json...
✔ 1186239 documents
✔ 13330 labels (max index 13329)

📝 Résumé :
- Min labels/doc : 1
- Max : 57
- Moyenne : 5.04
- Min docs/classe : 1
- Max : 355211
- Moyenne : 448.57
- Médiane : 28.00
- Classes < 10 docs : 3936
- Classes > 1000 docs : 718
- Classes jamais utilisées : 0

🏷️ Top 10 classes les plus fréquentes :
- Classe 1471 : 355211 docs → books
- Classe 7961 : 194561 docs → music
- Classe 7892 : 128026 docs → movies & tv
- Classe 9237 : 120090 docs → pop
- Classe 7083 : 97803 docs → literature & fiction
- Classe 7891 : 88967 docs → movies
- Classe 4038 : 76277 docs → education & reference
- Classe 10063 : 75035 docs → rock
- Classe 12630 : 71667 docs → used & rental textbooks
- Classe 8108 : 71667 docs → new

📌 Rappel :
- Les étiquettes sont extraites de trn.json (champ 'target_ind')
- Les noms de classes sont issus de Yf.txt (ligne = nom)
```

Figure : Analyse de la distribution des classes (labels)



Figures : Diagrammes en baton représentant la distribution des classes de notre dataset

Le constat est clair :

- Chaque document n'a en moyenne que 5 étiquettes sur 13 330 possibles -> largement < 0.1 % activées.
- Cela confirme que le matrice Y (labels) est très creuse.

-> **Sparsité élevée des étiquettes**

De plus :

- La médiane de 28 docs/classe est très inférieure à la moyenne (448), ce qui indique un long tail (grande majorité de classes peu représentées).

- ~ 30% des classes sont très rares (< 10 documents).
- Quelques classes très fréquentes (> 100k docs) faussent la moyenne → distribution fortement skewed vers la droite (long tail).

-> **Distribution très déséquilibrée entre les classes**

Récap :

- La majorité des classes ont peu de documents
- Quelques classes ont énormément de documents

Objectif : Bien gérer les classes rares

On va donc d'abord affiner (fine-tuning) notre modèle en l'entraînant sur notre dataset puis noter ses performances. Puis nous allons comparer cela avec première approche d'échantillonnage basé sur la fréquence des labels (du plus au moins fréquent).

Mais avant cela on va tout de même expliquer sur quelles métriques nous nous sommes basés pour évaluer les performances de nos modèles :

P@k (Precision@k) – proportion de bons labels parmi les k premiers prédits pour chaque document.

Micro vs Macro – *Micro* agrège sur toutes les paires document-label (met l'accent sur les classes fréquentes) ; *Macro* calcule la moyenne des métriques par label (pondération égale, met en valeur les rares).

P propensity-scored PSP@k – variante où chaque label est re-pondéré par un facteur de propension proportionnel à l'inverse de la fréquence ; elle corrige le biais vers les classes fréquentes et mesurent la qualité sur les labels rares.

PSnDCG@k (normalized Discounted Cumulative Gain) – même idée que P@k mais pondère la position : un label correct placé plus haut vaut plus, puis le score est normalisé entre 0 et 1.

Key: tp = True Positive, tn = True Negative, fp = False Positive, fn = False Negative			
Metric Name	Metric Formula	Code	When to use
Accuracy	$\text{Accuracy} = \frac{tp + tn}{tp + tn + fp + fn}$	tf.keras.metrics.Accuracy() or sklearn.metrics.accuracy_score()	Default metric for classification problems. Not the best for imbalanced classes.
Precision	$\text{Precision} = \frac{tp}{tp + fp}$	tf.keras.metrics.Precision() or sklearn.metrics.precision_score()	Higher precision leads to less false positives.
Recall	$\text{Recall} = \frac{tp}{tp + fn}$	tf.keras.metrics.Recall() or sklearn.metrics.recall_score()	Higher recall leads to less false negatives.
F1-score	$\text{F1-score} = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$	sklearn.metrics.f1_score()	Combination of precision and recall, usually a good overall metric for a classification model.
Confusion matrix	NA	Custom function or sklearn.metrics.confusion_matrix()	When comparing predictions to truth labels to see where model gets confused. Can be hard to use with large numbers of classes.

Figure : Définition des métriques de précision, rappel, et F1

Nous allons maintenant définir quels sont les hyperparamètres que l'on a utilisé pour procéder à l'entraînement :

```
srn python3 -m pecos.xmc.xtransformer.train \
  -t datasets/AmazonCat-13K/train_raw_texts.txt \
  -x datasets/AmazonCat-13K/X.trn.npz \
  -y datasets/AmazonCat-13K/Y.trn.npz \
  -m /data1/benamar/xrt/model_AmazonCat13K_2-ot-precalc \
  --batch-size 512 \
  --gradient-accumulation-steps 2 \
  --num-train-epochs 3 \
  --max-leaf-size 200 \
  --max-match-clusters 512 \
  --seed 2
```

Explication des hyperparamètres

- `-t datasets/.../train_raw_texts.txt`
→ Fichier contenant les textes bruts des documents (titres, descriptions).
- `-x datasets/.../X.trn.npz`
→ Matrice des features (représentation vectorielle des documents).
- `-y datasets/.../Y.trn.npz`
→ Matrice des labels associés aux documents (multi-label binaire).
- `-m /data1/.../model_AmazonCat13K_2-ot-precalc`
→ Dossier de sortie du modèle entraîné (où seront sauvegardés les poids et la config).
- `--batch-size 512`
→ Nombre de documents traités en parallèle avant une mise à jour des poids du modèle. Ici, 512 exemples par batch.
- `--gradient-accumulation-steps 2`
→ Permet de cumuler les gradients sur 2 batchs avant de faire une mise à jour des poids. Cela équivaut à un batch effectif de $512 \times 2 = 1024$, utile si la mémoire GPU est limitée.
- `--num-train-epochs 3`
→ Nombre de fois que le modèle va parcourir l'ensemble du dataset pendant l'entraînement.
- `--max-leaf-size 200`
→ Taille maximale d'un cluster feuille dans l'arbre HLT.
- `--max-match-clusters 512`
→ Nombre maximum de clusters candidats considérés lors de la prédiction.
- `--seed 2`
→ Valeur de la graine aléatoire qui définit l'ordre de tirage des documents. Nous avons, à chaque entraînement, lancés trois runs avec trois valeurs de seeds différentes afin de pouvoir moyenner nos métriques et diminuer/faire disparaître ce caractère aléatoire.

Je souhaite préciser également que l'échantillonneur (sampler) par défaut de pecos est uniforme et donc cela veut dire que chaque label a la même probabilité d'être choisi au moment de l'échantillonnage.

Après avoir entraîné notre modèle sur notre dataset (avec le sampler par défaut), voici les résultats :

k	Micro P	Micro R	PSP	PS-nDCG	Macro P	Macro R	Macro F1
1	95.26%	27.09%	95.66%	95.66%	11.05%	1.20%	2.16%
3	81.78%	61.96%	82.44%	91.11%	29.73%	10.27%	15.26%
5	66.36%	77.46%	67.20%	89.34%	43.37%	33.81%	38.00%

Micro P@k (chute de 95.3→81.8→66.4 %) : le modèle est bon sur ses premières prédictions, mais en ajoutant des rangs supérieurs, il introduit davantage de faux positifs, d'où la baisse de précision.

Micro R@k (montée de 27.1→62.0→77.5 %) : plus k augmente, on a naturellement plus d'occasions de capturer des labels pertinents, ce qui fait monter le rappel, ce qui fait que le modèle couvre progressivement une part importante des vrais labels.

PSP@k chute de 95.7→82.4→67.2 % : même en pondérant plus fort les labels rares, la qualité moyenne des k premiers prédits diminue quand k augmente.

PS-nDCG@k reste très haut (95.7→91.1→89.3 %) : le classement des vrais labels est plutôt bon, les erreurs en fin de liste sont moins pénalisées.

Macro-Precision-Label@k grimpe de 11.0→29.7→43.4 % : plus k monte, meilleure est la couverture uniforme des labels, mais la longue traîne (tail) reste difficile à capturer pleinement.

Macro-Recall-Label@k (montée de 1.2 → 10.3 → 33.8 %) : la plupart des labels, en particulier les rares, ne sont jamais capturés en top-1, d'où un recall macro très bas à 1.2 %. En passant à k=3 puis 5, on offre plus d'opportunités d'atteindre ces labels difficiles, ce qui fait bondir la couverture moyenne, mais elle reste loin du rappel micro, illustrant la difficulté d'atteindre uniformément tous les labels dans un espace extrême.

Plus k augmente, on couvre mieux chaque label (R augmente) tout en gardant une précision raisonnable (P augmente), d'où la progression monotone de **Macro-F1**.

Après avoir vu les performances avec notre sampler par défaut (uniforme), c'est là qu'entre en jeu notre première proposition.

Remarque :

Sur le papier, pour le même dataset, on obtient P@5 = 68.04

Or pour nous, P@5 = 66.36

Cela s'explique par le fait que nous n'avons pas utilisé exactement les mêmes hyper-paramètres que les concepteurs de pecos, notamment num-train-epochs (3 au lieu de 5).

Avec num-train-epochs = 5 , cela nous aurait pris plusieurs jours pour l'entraînement sur le A40

Avec 4 ça aurait pris un temps considérable également

num-train-epochs = 3 est le bon compromis que nous avons réussi à trouver entre temps d'entraînement raisonnable et baisse de la précision acceptable.

Première approche :

A présent, nous allons expliquer la philosophie derrière notre nouveau sampler « fait-maison » basé sur la fréquence des labels.

L'idée de départ a été de distinguer les labels fréquents (que j'ai appelés head) des labels rares (tail). Concrètement, j'ai commencé par compter combien de documents étaient associés à chaque étiquette. Celles qui apparaissaient dans un grand nombre de documents (plus de 400) étaient considérées comme head, et les autres comme tail.

Ensuite, pour chaque document i , j'ai calculé deux valeurs : le nombre de labels head et le nombre de labels tail qu'il contenait. Cela m'a permis de construire deux distributions de probabilités :

- p_i , proportionnel au nombre de labels « head » (fréquents) du document i
- q_i , proportionnel au nombre de labels « tail » du document i

Au moment de l'échantillonnage, j'ai mélangé ces deux distributions à l'aide d'un coefficient λ dans la formule ci-dessous :

$$\pi_i = (1-\lambda) * p_i + \lambda * q_i \quad , \lambda \in [0,1]$$

Ce coefficient commence proche de 0 au début de l'entraînement (ce qui privilégie les documents head), puis augmente progressivement pour donner de plus en plus d'importance aux documents tail. Cela crée une sorte de curriculum automatique : au début, le modèle apprend surtout sur les classes fréquentes, puis il est progressivement exposé aux classes plus rares.

Ce choix n'est pas un tri strict : le tirage reste aléatoire, mais statistiquement, les documents avec des labels fréquents apparaissent surtout dans les premiers lots, tandis que ceux avec des labels rares se retrouvent plutôt vers la fin. Enfin, à chaque époque, l'ordre est régénéré : tous les documents sont vus une fois, mais toujours selon cette logique head $\rightarrow$ tail.

Grâce à ce mécanisme, le modèle bénéficie d'un apprentissage plus stable : il consolide d'abord ses connaissances sur les classes majoritaires avant de s'attaquer progressivement aux classes rares, ce qui améliore son équilibre entre précision globale (micro-F1) et capacité à gérer les labels peu représentés (macro-F1).

Voici les résultats finaux suite à notre entraînement à l'aide de notre nouveau sampler :

k	Micro P	Micro R	PSP	PS-nDCG	Macro P	Macro R	Macro F1
1	95.61%	27.24%	96.01%	96.01%	14.07%	1.52%	2.74%
3	81.80%	62.01%	82.50%	91.27%	35.62%	13.10%	19.16%
5	66.37%	77.53%	67.23%	89.48%	45.28%	36.83%	40.62%

Figure : performances du nouveau sampler basé sur la fréquence des labels

k	Micro P	Micro R	PSP	PS-nDCG	Macro P	Macro R	Macro F1
1	95.26%	27.09%	95.66%	95.66%	11.05%	1.20%	2.16%
3	81.78%	61.96%	82.44%	91.11%	29.73%	10.27%	15.26%
5	66.36%	77.46%	67.20%	89.34%	43.37%	33.81%	38.00%

Figure : performances du sampler uniforme (par défaut)

On observe une nette amélioration sur absolument toutes les métriques ce qui montre l'efficacité du nouveau sampler dont l'échantillonnage est basé sur la fréquence des labels.

Nous avons également réalisé une version avec tri parfait déterministe (donc sans échantillonnage avec lambda) et voila les résultats que l'on a obtenu :

k	Micro P	Micro R	PSP	PS-nDCG	Macro P	Macro R	Macro F1
1	95.68%	27.27%	96.07%	96.07%	13.26%	1.39%	2.52%
3	81.88%	62.05%	82.58%	91.34%	36.15%	13.15%	19.29%
5	66.44%	77.57%	67.30%	89.54%	45.88%	37.32%	41.16%

Figure : performances du modèle avec un sampler déterministe (sans aucune stochasticité)

Dans cette version, tri parfait déterministe (sans utiliser lambda, précalcul du curriculum à l'avance). C'est là qu'on observe les meilleures métriques comparé à la baseline.

Remarque :

Avant d'en arriver là, il a fallu lire les papiers, et tester plusieurs approches qui n'ont pas abouti ce qui a pris un temps considérable.

Maintenant que l'on a vu l'efficacité de l'échantillonnage basé sur la fréquence des labels, nous allons essayer d'aller encore plus loin en essayant une nouvelle approche : le transport optimal, qui lui, prend également en compte la sémantique derrière chaque label.

2. Deuxième approche: échantillonnage basé sur le transport optimal

Le principe du transport optimal vient des mathématiques appliquées : il s'agit de trouver le moyen le plus efficace de transformer une distribution en une autre, en minimisant un "coût de transport". Dans notre contexte, l'idée est de définir un plan de transport qui permette de relier les documents aux labels en tenant compte à la fois de leur fréquence et de leur proximité dans l'espace des représentations.

Pour mesurer cette similarité, nous utilisons les embeddings des labels (vecteurs qui représentent leur sens). C'est avec la distance cosinus que l'on va mesurer leur proximité : deux labels avec des embeddings très proches auront une distance faible, ce qui signifie qu'ils sont sémantiquement liés (par exemple « montre » et « horloge »).

En effet, voici comment fonctionne le calcul de cette distance (Source : Wikipedia) :

Soit deux vecteurs A et B , le cosinus de leur angle θ s'obtient en prenant leur [produit scalaire](#) divisé par le produit de leur [norme](#) :

$$\cos \theta = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}.$$

La valeur d'un cosinus, donc celle calculée ici pour $\cos \theta$, est comprise dans l'intervalle $[-1,1]$. La valeur de -1 indique des vecteurs opposés, la valeur de 0 des vecteurs indépendants (orthogonaux) et la valeur de 1 des vecteurs [colinéaires](#) de coefficient positif. Les valeurs intermédiaires permettent d'évaluer le degré de similarité.

Maintenant, nous allons définir les éléments que nous avons utilisé dans cette partie :

1. La matrice de coûts C

La matrice C de taille $L \times L$ contient le coût pour passer d'un label l à un autre l' . Ici, ce coût est défini par :

$$C[l, l'] = 1 - \cos(e_l, e_{l'})$$

où e_l et $e_{l'}$ sont les embeddings des labels. Deux labels proches sémantiquement (cosinus élevé) auront donc un coût faible, tandis que deux labels éloignés auront un coût élevé.

2. Le noyau K

Pour stabiliser le calcul, on applique une régularisation entropique et on définit un noyau de Gibbs :

$$K = \exp(-C / \text{epsilon})$$

Ce noyau transforme les coûts en similarités : une faible distance cosinus donne une valeur élevée dans K .

3. Le plan de transport Γ

Le transport optimal cherche une matrice Γ qui indique combien de probabilité "transférer" de chaque label l vers un label l' . On calcule Γ via l'algorithme de Sinkhorn, qui ajuste deux vecteurs de normalisation (u, v) jusqu'à ce que la matrice respecte les distributions de départ et d'arrivée $(p$ et $q)$.

4. Le rôle de epsilon

Le paramètre epsilon règle la régularisation :

- Si epsilon est petit, le transport est très précis mais instable.
- Si epsilon est plus grand, le transport est plus "lisse" : la masse se répartit sur plusieurs labels proches, ce qui est souvent préférable pour l'apprentissage.

5. Le barycentre de Sinkhorn

Pour créer un curriculum progressif, on mélange les deux distributions :

- p : centrée sur les labels fréquents (head).
- q : orientée vers les labels rares (tail).

Le barycentre régularisé est défini par :

$$p_{\lambda} = \text{bary_epsilon}^{\lambda}(p, q)$$

- Quand $\lambda = 0$, on reste sur les labels fréquents (head).
- Quand $\lambda = 1$, on se rapproche des labels rares (tail).

- En augmentant progressivement λ , on suit une géodésique de Wasserstein, c'est-à-dire un chemin optimal entre head et tail.

Remarque 1:

J'ai fait des tentatives pour cette partie là mais rien de concluant pour l'instant. Je compte continuer à travailler la-dessus jusqu'à obtenir des résultats concrets.

Remarque 2 :

Je n'ai pas eu le temps non plus d'aborder la partie meta-learning mais l'idée de base est que ce type d'approche permet au modèle d'apprendre à apprendre : au lieu de se limiter à une tâche fixe, il s'entraîne sur un ensemble de petites tâches variées, ce qui lui donne la capacité de s'adapter rapidement à de nouvelles classes ou à des labels rares avec très peu d'exemples (few-shot learning).

VII. Ce que ce stage m'a apporté

Ce stage m'a permis de me familiariser avec le monde de la recherche, et plus particulièrement dans un domaine qui m'intéresse tout particulièrement : le machine learning. Même si ce domaine était encore nouveau pour moi et que j'ai parfois rencontré des difficultés, cette expérience m'a servi de véritable introduction au machine learning et au traitement automatique du langage (NLP).

J'ai ainsi pu découvrir concrètement des concepts que je n'avais pas encore étudiés en cours. Avec le recul, j'aurais aimé que des enseignements comme « Fouille de textes » avec M. Hamon ou « Introduction au TAL et réseaux de neurones » avec M. Leroux soient proposés plus tôt dans mon parcours. Cela m'aurait certainement permis d'aborder ce stage avec davantage de bases théoriques. Mais cela fait partie du chemin d'apprentissage, et ce stage a justement comblé en partie ce manque.

Au-delà des aspects techniques, ce stage m'a également beaucoup appris sur le travail en autonomie : savoir chercher par moi-même, avancer pas à pas face aux difficultés, et organiser mon travail de recherche. Dans ce processus, les conseils de mon encadrant, M. Nadi Tomeh, ont été d'une aide précieuse : ses orientations et ses retours m'ont guidé efficacement tout au long de mon stage.

Conclusion

Pour conclure, ce stage de quatre mois au LIPN a été une expérience très enrichissante, à la fois sur le plan technique et personnel. J’y ai découvert le monde de la recherche et j’ai pu m’initier concrètement au machine learning et au traitement automatique du langage, deux domaines qui m’intéressent beaucoup.

Au-delà des aspects scientifiques, ce stage m’a aussi appris à travailler de façon plus autonome, à chercher par moi-même et à avancer pas à pas face aux difficultés. Les conseils de mon encadrant, M. Nadi Tomeh, m’ont beaucoup aidé et m’ont montré l’importance de la discussion, de la collaboration et du partage dans un projet de recherche.

Je garde de cette expérience non seulement des connaissances techniques, mais aussi une meilleure compréhension de ce qu’implique la recherche au quotidien. Cela me donne envie de continuer à approfondir mes compétences et me motive encore plus pour la suite de mon parcours. Après cette immersion dans la recherche, je souhaite maintenant profiter de cette année pour voir comment cela se passe en entreprise, afin de comparer les deux environnements et mieux orienter mes choix futurs.

References

- Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In Proceedings of the 26th Annual International Conference on Machine Learning, ICML ’09, page 41–48, New York, NY, USA, 2009. Association for Computing Machinery. ISBN 9781605585161. doi: 10.1145/1553374.1553380.
URL <https://doi.org/10.1145/1553374.1553380>.
- Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transport. In C. J. Burges, L. Bottou, M. Welling, Z. Ghahramani, and K. Q. Weinberger, editors, Advances in Neural Information Processing Systems, volume 26. Curran Associates, Inc., 2013.
URL https://proceedings.neurips.cc/paper_files/paper/2013/file/af21d0c97db2e27e13572cbf59eb343d-Paper.pdf.
- Kunal Dahiya, Diego Ortego, and David Jimenez-Cabello. Prototypical extreme multi-label classification with a dynamic margin loss. In Luis Chiruzzo, Alan Ritter, and Lu Wang, editors, Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), pages 10709–10727, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. ISBN 979-8-89176-189-6.
URL <https://aclanthology.org/2025.naacl-long.537/>.
- Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. CoRR, abs/1703.03400, 2017.
URL <http://arxiv.org/abs/1703.03400>.

- Ting Jiang, Deqing Wang, Leilei Sun, Huayi Yang, Zhengyang Zhao, and Fuzhen Zhuang. Lightxml: Transformer with dynamic negative sampling for high-performance extreme multi-label text classification. Proceedings of the AAAI Conference on Artificial Intelligence, 35(9):7987–7994, May 2021. doi: 10.1609/aaai.v35i9.16974.
URL <https://ojs.aaai.org/index.php/AAAI/article/view/16974>.
- Xintong Li, Jinya Jiang, Ria Dharmani, Jayanth Srinivasa, Gaowen Liu, and Jingbo Shang. Open-world multi-label text classification with extremely weak supervision, 2024.
URL <https://arxiv.org/abs/2407.05609>.
- A. Mittal, N. Sachdeva, S. Agrawal, S. Agarwal, P. Kar, and M. Varma. Eclare: Extreme classification with label graph correlations. In Proceedings of The ACM International World Wide Web Conference, April 2021.
- Mohammad Pezeshki, Sékou-Oumar Kaba, Yoshua Bengio, Aaron Courville, Doina Precup, and Guillaume Lajoie. Gradient starvation: A learning proclivity in neural networks. In A. Beygelzimer, Y. Dauphin, P. Liang, and J. Wortman Vaughan, editors, Advances in Neural Information Processing Systems, 2021.
URL <https://openreview.net/forum?id=aExAsh1UHZo>.
- Anthony Rios and Ramakanth Kavuluru. Few-shot and zero-shot multi-label learning for structured label spaces. In Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun’ichi Tsujii, editors, Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, pages 3132–3142, Brussels, Belgium, October-November 2018. Association for Computational Linguistics. doi: 10.18653/v1/D18-1352.
URL <https://aclanthology.org/D18-1352/>.
- Daniel Simig, Fabio Petroni, Pouya Yanki, Kashyap Popat, Christina Du, Sebastian Riedel, and Majid Yazdani. Open vocabulary extreme classification using generative models. In Smaranda Muresan, Preslav Nakov, and Aline Villavicencio, editors, Findings of the Association for Computational Linguistics: ACL 2022, pages 1561–1583, Dublin, Ireland, May 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.findings-acl.123.
URL <https://aclanthology.org/2022.findings-acl.123/>.
- Jake Snell, Kevin Swersky, and Richard S. Zemel. Prototypical networks for few-shot learning. CoRR, abs/1703.05175, 2017.
URL <http://arxiv.org/abs/1703.05175>.
- Yibo Wang, Jun-Yi Hang, and Min-Ling Zhang. Multi-label open set recognition. In The Thirty-eighth Annual Conference on Neural Information Processing Systems, 2024.
URL <https://openreview.net/forum?id=k1VrxRS6WZ>.
- Jiong Zhang, Wei-Cheng Chang, Hsiang-Fu Yu, and Inderjit S Dhillon. Fast multi-resolution transformer fine-tuning for extreme multi-label text classification. In A. Beygelzimer, Y. Dauphin, P. Liang, and J. Wortman Vaughan, editors, Advances in Neural Information Processing Systems, 2021.
URL <https://openreview.net/forum?id=gjBz22V93a>.

Lien pour récupérer le dataset : <http://manikvarma.org/downloads/XC/XMLRepository.html>

Lien github du modèle XR-Transformer :

<https://github.com/amzn/pecos/blob/mainline/pecos/xmc/xtransformer/README.md>